Atlas AI Training — Trainer Preparation Guide

Atlas AI Training — Trainer Preparation Guide

This document is for the trainer only. It covers what to set up ahead of the session, what to know deeply, common participant questions, and what was found from a full review of the Cognite Docs that participants may ask about.

Trainer docs: Atlas AI | Concepts | Build agents | Prompting | Language models | Agent tools | Runtime versions | Python tool | Call Function tool | Evaluation overview | Evaluate agents | Optimizing data models for AI | Capabilities

---

1. Pre-session checklist (complete 2-3 days before)

CDF and Atlas AI environment

- [ ] Confirm every participant has a CDF login and Atlas AI is enabled (separate license required).
- [ ] Confirm participants have the following CDF capabilities:
  - `agents:read`, `agents:write`, `agents:run` — to build and interact with agents.
  - `datamodels:read` (space-scoped) — required when using query tools for data retrieval.
  - `datamodelinstances:read` (space-scoped) — required when using query tools.
  - Note: additional capabilities may be required depending on tools enabled.
- [ ] Confirm participants can navigate to **Atlas AI > Agent builder** and **Atlas AI > Evaluate agents** in CDF.
- [ ] Confirm at least one data model/view is available to use for query tool demos. Know the space, data model external ID, and view name.
- [ ] Identify 2-3 example Streamlit agents or published agents in the Agent library to demonstrate.

Pre-built demo agent (highly recommended)

- [ ] Build a demo agent in the shared CDF project before the session so you can show a working example without building from scratch live.
- [ ] Include in the demo agent:
  - Clear instructions with role, output format, and constraints.
  - At least one query tool (e.g. Find assets or Find maintenance orders) configured with the correct data model/view/scope.
  - 3 sample prompts that show the range from vague to highly specific.
- [ ] Pre-run the demo agent with 3-5 prompts and confirm responses are as expected.

Pre-built evaluation

- [ ] Create an evaluation in **Atlas AI > Evaluate agents** with 4-5 test cases ready to run live.
- [ ] Include at least one edge case (missing data, ambiguous equipment ID) to show how the agent responds.
- [ ] Know which test cases will pass and which may struggle — this makes for realistic discussion.

Slides and materials

- [ ] Export slides from `slidev/` (`npm run export:pptx`) and confirm PPTX is in `docs/slides/`.

- [ ] Have the training agenda (`docs/atlas-ai-training-agenda.md`) open for reference.
- [ ] Bookmark all doc links in this guide in your browser.

---

2. Topics from full documentation review — what we covered and gaps

The docs MCP search confirmed the following **topics that are covered** in the training: - Agent overview, language models, knowledge graph, prompts, tools, runtime versions. - Build and publish workflow. - Effective prompting: specificity, context, output format, reasoning/tool call traces. - Evaluation: design, run, interpret, iterate.

Topics found in docs that add useful depth — consider mentioning in session:

Access control and capabilities

Source: Capabilities - To build: `agents:read`, `agents:write`, `agents:run` - To interact with agents: `agents:run`, `agents:read` - For data retrieval tools: `datamodels:read`, `datamodelinstances:read` (space-scoped) - Agent-level access control: grant read/write/run per agent, control visibility. - Teams can collaborate and co-maintain agents in a group. - **Trainer note:** Mention when participants ask “why can’t I see the agent?” or “why is query returning nothing?”

Conversation history and compliance

Source: Prompting - All conversations are saved as records. Participants can review, continue, and reuse past conversations. - Conversation history stores reasoning and referenced data for compliance and safety audit traceability. - **Trainer note:** Good to mention for industrial/regulated environments; supports the “why AI decisions are traceable” point.

Runtime versions — upgrade process

Source: Agent runtime versions - Each agent is pinned to a specific runtime version at creation time. - Runtime includes: system prompt, tool set, and feature set. - Before upgrading: review feature differences, test in non-production, verify custom tools/integrations. - Auto-upgrade happens at retirement date if not manually upgraded. - **Trainer note:** Relevant for production-deployed agents. Emphasize testing before upgrading.

Run Python code tool — how it works

Source: Run Python code - The tool validates code, generates a JSON schema from Python type annotations, and passes it to the LLM as a tool descriptor. - The agent reads parameter docstrings (Args: section) to understand what values to pass — **docstrings directly affect agent behavior**. - Includes a `test()` function you can run from the Agent builder UI to validate the tool before deployment. - **Trainer note:** This is an advanced topic; mention as a stretch. Useful for teams that want custom calculations.

Call Function tool — configuration details

Source: Call Function - Calls a deployed Cognite Function (pre-deployed Python code) from an agent. - Requires: tool name, tool instructions (when/how to use), function name, max polling time (default 540s), JSON Schema for arguments. - One Call Function tool instance = one function. Add separate tools for each function. - **Trainer note:** Useful for teams already using Cognite Functions for ETL/calculations. Show as a “next step” option.

Optimizing data models for AI

Source: Optimizing data models for AI - Agents have context about the underlying data model, including **documentation in the data model definition** — well-documented models produce better agent responses. - Key concepts to model properly for agent grounding: Assets (physical hierarchy), Time series (linked to assets, clear units), Files (linked to assets/equipment), Activities (typed, scoped maintenance/events), Relations (consistent asset-to-file, activity-to-equipment). - Consistent relationships enable multi-hop queries and semantic linking. - **Trainer note:** Very relevant for the "why does my agent give wrong results?" conversation. Poor data model documentation = poor agent grounding.

Agents in Canvas and Charts (user-facing surfaces)

Source: Use an agent to find data (Canvas) and Charts - Users can invoke agents from Canvas (add results directly to canvas) and Charts (analyze time series). - Location scope: users select which location/space the agent queries. - **Trainer note:** Good to show if participants are end-users as well as builders. Agents aren't only available in the Atlas AI workspace.

Agent library and templates

Source: Atlas AI, Build agents - Agents can be created from scratch or from templates. Templates provide a starting point for common industrial use cases. - Published agents appear in the **Agent library** and are available to all CDF project users. - **Trainer note:** Point participants to templates at the start of the build exercise — faster onboarding.

---

3. Key concepts to know deeply (trainer background)

Why language model choice matters

- Different models have different context window sizes (how much data they can reason over), response quality, and latency.
- Retirement dates mean production agents need a lifecycle plan — if not migrated, auto-upgrade at retirement.
- Model availability depends on cloud vendor (Azure, GCP, AWS) and CDF project region.

How tools shape agent behavior

- Enabling only the tools needed for a use case improves response quality and prevents unintended tool selection.
- For query tools, the agent uses the data model + view you configure to scope its knowledge graph queries.
- Access scope options: inherit from user's location (uses CDF location setting), inherit from user's access rights (no space filter), or manually defined spaces.

How the reasoning trace works

- When an agent response is unexpected, the reasoning field shows the agent's decision path: how it interpreted the prompt, which tools it selected, and what parameters it used.
- Tool call inputs/outputs show exactly what data was queried and returned.
- This is the primary debugging mechanism — train participants to use it rather than guessing.

What makes a good expected response in evaluations

- Must be specific enough that pass/fail is clear.
- Should name the exact data points, status values, or fields the agent must include.

- Example: “Pump P-101 is operational and running at 85% capacity with no active alerts” — not “Pump P-101 is running fine.”

---

4. Common participant questions — prepared answers

Question	Answer
“Why can’t I see Atlas AI in my CDF?”	Atlas AI requires a separate license. Contact Cognite or your CDF admin.
“Why is the query tool returning nothing?”	Check: data model/view configured correctly, correct space scope, participant has `datamodels:read` and `datamodelinstances:read` for that space.
“Can I use multiple data models in one agent?”	Yes — add one query tool per data model/view. Each tool can be configured with its own data model and scope.
“What happens if I don’t upgrade my agent’s runtime?”	Atlas AI will auto-upgrade it to the latest stable version when the pinned runtime reaches its retirement date. Test before then.
“Can agents write or update data?”	Query tools are read-only. Integration tools (Call REST API, Run Python code, Call Function) can potentially write, depending on how they’re implemented.
“How do I reuse good prompts?”	Save them from conversation history. You can review past conversations and use successful prompts as templates.
“Is conversation history shared?”	Conversations are stored per user. Reasoning and data references are preserved for compliance audits.
“How many tools should I enable?”	Only the tools needed for your specific use case. More tools increases the chance the agent picks the wrong tool.
“Can the agent call external APIs?”	Yes — via the Call REST API integration tool (beta). You can call Cognite API endpoints with GET/POST.
“How do I build a custom calculation tool?”	Use Run Python code (inline) or Call Function (pre-deployed Cognite Function). Both use JSON Schema to tell the agent what arguments to pass.
“Does the agent understand my data model?”	The agent reads data model documentation. Well-documented fields, relationships, and descriptions in the data model definition improve agent accuracy.

---

5. Session timing guide

Segment	Time	Trainer actions
Audience intro + goal	5 min	Orient participants; confirm everyone has CDF access
Atlas AI overview and concepts	15 min	Lecture with slides; draw the agent stack diagram
Language models and tools reference	15 min	Walk through model lifecycle table; demo tool categories in Agent builder

Segment	Time	Trainer actions
Build an agent — demo	10 min	Show the pre-built demo agent; walk through instructions and tool config
Build an agent — hands-on	10 min	Participants build or edit an agent; circulate and assist
Prompting — demo and examples	20 min	Live demo: vague → specific → highly specific; show reasoning trace
Evaluation — create and run	20 min	Create evaluation live; run against demo agent; walk through results
Evaluation — hands-on	15 min	Participants add test cases and run evaluations
Wrap-up and Q&A	10 min	Recap loop; surface next steps; open Q&A

---

6. Whiteboard / visual to prepare

Draw this on a whiteboard or blank slide at the start:

```
User prompt
     │
     ▼
┌──────────────────────────────────┐
│          Atlas AI Agent          │
│  Language model (reasoning)      │
│  Instructions + Goals            │
│  Tools (query / analysis / code) │
│  Runtime version (pinned)        │
└──────────────────────────────────┘
                │
                ▼
        CDF Knowledge Graph
        (data model + instances
         + time series + files)
```

Explain that the agent's quality is a product of: the model you choose, the instructions you write, the tools you enable, and the quality of the underlying data model.

---

7. What to say about topics we don't cover in depth

Topic	What to say
Agent APIs (alpha)	"Cognite has APIs to embed agents in third-party apps like Streamlit — alpha. Ask your Cognite contact."
Agent-level access control	"You can control who can see/run/edit each agent by configuring per-agent permissions. See your CDF admin."
Data model optimization	"If your agent gives wrong results, the data model documentation may be the root cause. See the 'Optimizing data models for AI' guide."
Integration tools in production	"REST API, Python code, and Call Function tools are in beta. Test thoroughly and check for updates before using in production."

Topic	What to say
Conversation history and compliance	“Every agent conversation stores reasoning and data references. This supports audit and compliance workflows.”

---

8. Reference links (quick access on training day)

- Atlas AI overview
- About Atlas AI agents (concepts)
- Build and publish agents
- About prompting Atlas AI agents
- Language model library
- Agent tools library
- Agent runtime versions
- Run Python code tool
- Call Function tool
- About AI agent evaluations
- Evaluate Atlas AI agents
- Optimizing data models for AI
- CDF capabilities for agents
- Use agents in Canvas
- Use agents in Charts